AI Evaluation Engineering

Foundations, and the system you are about to evaluate



Toni Ramchandani

https://www.linkedin.com/in/toni-ramchandani/

How a model answers · What evaluation is · Framing and quality · Architecture and traces

Three modules, one running example

01

Foundations

How a model produces an answer. What an LLM agent is made of. What evaluation is, why it is hard, and the loop.

You leave with a named gulf for your own system

02

Framing and quality

Goals, objects, tasks, dimensions. Absolute against comparative. What counts as good, written down.

You leave with a drafted Evaluation Charter

03

Architecture and traces

Retrieval, the full request path, and the record of what the system actually did on each request.

You leave with six traces and four localised failures

Nothing is measured today. Day 1 produces the three things the other three days consume: a shared model of how an agent answers, an agreed vocabulary for quality, and traces that exist.

01

Foundations of AI Evaluation Engineering

Shared ground before any metric

Before we can evaluate anything we need to agree what we are evaluating. The first half of this module is how a model produces an answer and what an LLM agent is made of. The second half is what evaluation is, why it is hard, and the loop we will run for four days.

CASE ONE

The assistant contradicted its own source of truth

What the chatbot told Jake Moffatt

Already flown? Claim the reduced bereavement rate within 90 days of the date your ticket was issued.

What Air Canada's own policy page said

The bereavement policy does not cover requests made after travel is complete.

The tribunal found negligent misrepresentation, and rejected Air Canada's argument that the chatbot was a separate legal entity responsible for its own actions. Air Canada was ordered to pay $812.02 in damages, interest and fees.

Nobody caught the discrepancy before a customer relied on it. The distance between those two boxes has a name we will use for four days: **groundedness**.

CASE TWO

Purpose-built RAG products, marketed as hallucination-free

17–33% of queries answered with content that was incorrect or misgrounded, across three leading AI legal research tools (Magesh et al., 2025).

Naive retrieval	Inapplicable authority	Reasoning errors	Sycophancy
the right source was never found	wrong jurisdiction, or overruled	misread the retrieved text	accepted a false premise

RAG reduces hallucination. It does not remove it.

THE RUNNING EXAMPLE · ALL FOUR DAYS

Helix, an internal policy assistant

Corpus

An employee handbook and HR policy wiki

Users

Employees across five country offices

Stakes

Wrong answers create legal exposure and erode trust

State

Built, shipped, and broken in four documented ways

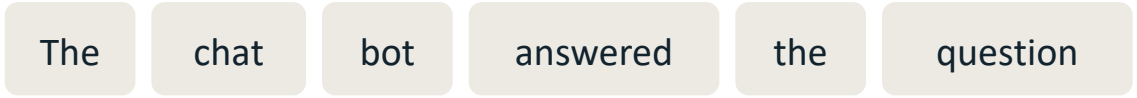
THE QUESTION WE KEEP COMING BACK TO

“How much parental leave am I entitled to at the Amsterdam office?”

1.1 HOW A MODEL PRODUCES AN ANSWER

One token at a time, each one a prediction

A **token** is the smallest unit of text a model reads or writes, roughly a word or part of one. “The chatbot answered the question” becomes:



The model was trained on one objective: **predict the next token**. Everything else (following instructions, citing a source, refusing) is behaviour learned on top of that objective, not a guarantee built into it.

Cost is counted in tokens in and tokens out, which is why context budgets exist and why something always gets left out.

Nothing in that objective says the output must be true, or grounded in the document you supplied.

Two consequences you will design around all week

Outputs are not repeatable

At the default temperature the same input can produce different outputs on different runs. Set **temperature 0** when evaluating, or sample repeatedly and report an interval.

This is why assert-based testing breaks here: there is no single correct string to compare against.

The model only ever sees messages

Every provider takes the same shape: a list of messages, each with a **role** and **content**. Retrieved documents are not special: they are text you chose to put in a message.

Whatever did not make it into a message does not exist, as far as the model is concerned.

Role	What it carries
system	Standing behaviour and context for the whole exchange
user	The request, plus in RAG the retrieved passages you decided to include
assistant	The model's previous replies, replayed to maintain a conversation
tool	The result of a function the model asked to call

1.2 THREE LIFECYCLE STAGES

Evaluation happens at every stage, and means something different at each

Pretraining

A base model learns next-token prediction over an enormous corpus.

Measured by
perplexity, scaling-law analysis

By whom
frontier labs

Says nothing about whether the model is useful for any real task

Post-training

The provider steers it toward instruction-following and safer output.

Measured by
public benchmarks approximating user tasks

By whom
model providers

These are the headline leaderboard numbers

Application

You wire the model into a workflow with retrieval, tools and prompts.

Measured by
whatever you define, against your data

By whom
you

No public leaderboard covers this. It is the whole job.

Neither of the first two stages says anything about your application. A model leaderboard never saw your corpus, your users or your definition of a good answer.

1.3 ANATOMY OF AN LLM AGENT

Every layer you add buys capability and costs you a new way to fail

1	**Single call**	One prompt in, one response out. No memory, no tools.	The prompt is the only thing you control
2	**+ Conversation**	History replayed on every turn as alternating messages.	Context length, and drift across turns
3	**+ Retrieval**	Documents fetched from a corpus and inserted into the prompt.	Everything that can go wrong in a corpus and an index
4	**+ Tools**	The model asks for a function; the result returns as a message.	Wrong tool, wrong arguments, irreversible actions
5	**Agent loop**	The model decides each step, observes, and repeats until done.	The space of possible behaviours stops being enumerable

Helix lives at layer 3: complex enough to fail in interesting ways, simple enough that every failure traces to one stage. On Day 3 its agentic and graph variants run on the same index.

Autonomy is the design choice underneath all of this. The more decisions you hand the model, the larger the space you have to evaluate.

Autonomy is decided before you evaluate, and it decides how much there is to evaluate

Fixed sequence

The steps are hard-coded. The model fills in text and nothing else.

To evaluate: a finite set of paths

Constrained choice

The model picks among a known set of tools, in a bounded number of steps.

To evaluate: paths, plus tool choice and arguments

Full discretion

The model decides what to do at every turn, including when to stop.

To evaluate: a space that is no longer enumerable

The choice usually lives in one line of the prompt:

> “If anything is ambiguous, ask the user to clarify before proceeding.” proceeding.”

> “If anything is ambiguous, infer the most likely intent and continue.”

Same system, two different evaluation problems. Make the choice explicit and the agent becomes predictable enough to measure.

Tool servers add a moving part of their own: an agent that worked yesterday can break today because a tool schema changed, and permission scoping has to be verified rather than assumed.

1.4 WHAT EVALUATION IS

A definition worth holding to

> Evaluation is the systematic process of **understanding** and **measuring** quality in an LLM application.

Understanding comes first

The qualitative half: reading outputs, finding the patterns in how the system fails, and naming them. Skipped more often than any other step.

Measuring comes second

The quantitative half: how often each named failure actually occurs, and whether that rate is tolerable.

A good evaluation produces results that are **interpretable** and **actionable**. A number nobody can interpret and nobody can act on is not an evaluation result. It is a dashboard.

Why this is harder than testing software you have written before

No formal specification

There is no written contract for what the system should and should not do. Writing one is part of the job, not a prerequisite you were handed.

No single correct output

Two different summaries can both be right. Scoring against one reference penalises perfectly good answers.

Failure is not always visible

An answer can be factually right and still wrong for the user, or sound entirely convincing while being false.

Neighbour	The question it answers	What breaks if it stands in for evaluation
Testing	Does this behave as specified?	Assumes a known-correct output exists for every input. Coverage shrinks to the trivial cases
Benchmarking	How does this compare with others?	A score on someone else's corpus says little about yours, and contamination is hard to rule out
Monitoring	Is production behaving as it did yesterday?	Detects change, not wrongness. A system that has always been wrong looks perfectly healthy

Testing needs a specification you already have. Evaluation is what you do when writing that specification is part of the job.

1.5 WHERE EVALUATION PLUGS IN

Four places an evaluation actually runs

IN THE CRITICAL PATH

Guardrails

Checks that run before the answer reaches the user, so the system can block, retry or fall back.

Latency and false-positive rate become product decisions

ALONGSIDE

Monitoring

Continuous checks in the background that detect drift or degradation over time.

Catches change, not wrongness

ALONGSIDE

Improvement

Labelled examples, few-shot cases, and the failure list that motivates architectural change.

The output of Day 2 feeds this directly

ALONGSIDE

Model migration

Comparing versions so a change of model is a decision with evidence behind it.

Impossible without a pinned baseline

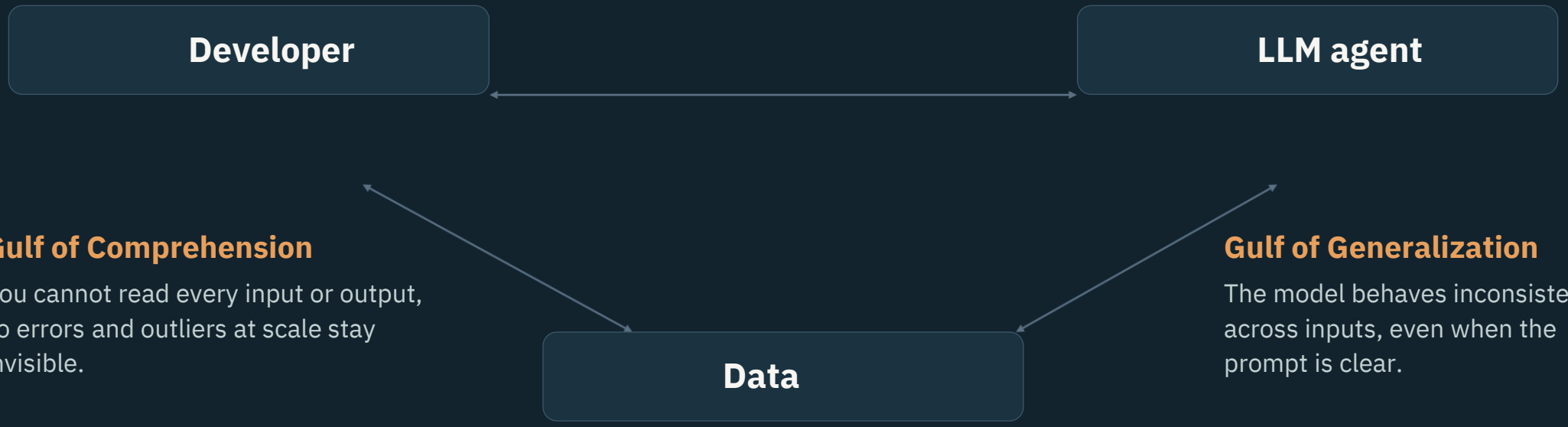
Only one of the four sits between the model and the user. The other three shape what you build next, which is why an evaluation that changes no decision is just overhead.

1.6 WHY THIS IS HARD

Three gulfs, three different things to do about them

Gulf of Specification

Your intent is only loosely captured by the prompt you wrote. Natural language is imprecise.



Gulf of Comprehension

You cannot read every input or output, so errors and outliers at scale stay invisible.

Gulf of Generalization

The model behaves inconsistently across inputs, even when the prompt is clear.

Each gulf is closed by a different activity. Adding instructions to the prompt is the universal reflex, and it narrows exactly one of the three.

Module 01 · 1.6 · Sources: Shankar et al. 2025 · Norman 1988

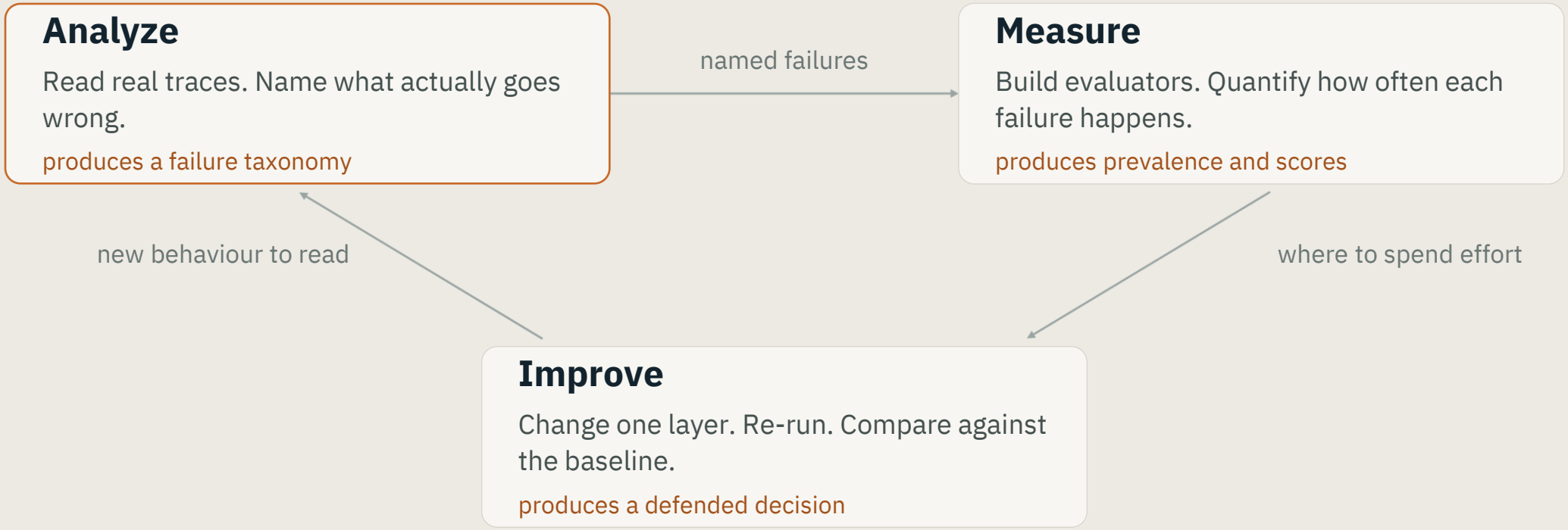
The same three gulfs, in Helix

Gulf	What it looks like here	What narrows it	Where we attack it
Comprehension	Nobody has read the corpus. Two versions of the parental leave policy are both indexed, and one of them was replaced in November 2024	Read traces. Inventory the corpus	Module 03 today Module 04 tomorrow
Specification	“Cite your sources” describes at least six different behaviours, and the prompt silently picked one	Write the task definition, the ground truth, and what the task is not	Module 02 today Module 05 tomorrow
Generalization	It works on the ten questions the team tried, and all ten came from the same two people	Define the input space and its coverage classes	Module 02 today Modules 04 and 09

> A failure mode is something the system *does*. A gulf is something you *cannot see*. That is why the day ends at traces rather than at metrics.

1.7 THE EVALUATION LIFECYCLE

You do not start at Measure



Skipping to Measure gives you a number you cannot act on: you cannot say which failures it covers, whether the value is acceptable, or what you would change to move it.

THE SHORTCUT THAT FEELS LIKE PROGRESS

Install the toolkit. Run the default metrics. Report faithfulness 0.82.

The number is not wrong. It is **unanchored**. You cannot say which of your failure modes it covers, whether 0.82 is acceptable here, or what you would change to move it.

Criteria do not survive first contact with real outputs either. People discover what they actually care about by grading things, which is why the charter you write next is a **v1**, and why tomorrow opens by revising it.

EXERCISE 1.A

Write alone, then compare in pairs

Name your gulf

1. In one sentence: the most recent thing you discovered about your own system that surprised you.

2. Classify it: comprehension, specification, or generalization?

Keep the sentence. Tomorrow morning you check whether error analysis confirms it or overturns it. Most of the time the thing people were sure about turns out not to be the thing that is actually hurting them.

02

Framing RAG evaluation and defining quality

Ends in a written charter

From here on the work is about your own system. Everything in this module is a decision somebody has to make and write down, and what you write goes into tomorrow.

2.1 MODEL QUALITY IS NOT SYSTEM QUALITY

The model is one component, and rarely the deciding one

The model

The trained engine that maps a list of messages to a continuation.

You choose it and prompt it. You do not change it.

The system

Ingestion · parsing · chunking · enrichment · embedding · indexing · query handling · retrieval · reranking · context assembly · prompting · generation · citation · guardrails · interface

You own almost all of it, and almost all of it can produce a wrong answer on its own.

An evaluation that varies only the model is measuring the wrong variable.

Position inside the prompt matters too: models use material at the beginning and end of a long context far more reliably than material buried in the middle. Context assembly order is a design decision with measurable consequences, and not one the model can fix for you.

SAME MODEL · SAME PROMPT · TWO CONFIGURATION CHANGES

“I am the primary caregiver in the Amsterdam office. How many working days of parental leave do I get?”

Ground truth: 130 working days, 26 weeks, primary caregiver, Amsterdam office · HB-PAR-2024 v4

Fixed chunks, headings dropped

All employees who become a parent are entitled to **12 weeks** of paid parental paid parental leave, equal to **60 working days**, at 60% of base salary. salary.
[HB-PAR-**2022**-00 v2 2022-03-15]

Answered from a policy retired in 2024. The figures are real, current for nobody.

Structural chunks, headings kept

Secondary caregiver, 10, 50, 100% for the first 6 weeks. Employees contracted to contracted to the Amsterdam office may spread the entitlement over the first the first twelve months.
[HB-PAR-**2024**-02 v4 2024-11-01]

Current policy now, wrong row still. The question said primary caregiver.

Two config changes fixed the staleness and left the wrong row untouched. Neither change went near the model. Both answers are fluent, specific and cited, and a user would have no way to tell either was wrong.

A prompt has parts, and each part fails differently

Part	What it does
Role	Anchors the model in the right domain
Objective	States the task and what success looks like
Instructions	The dos and don'ts, spelled out
Context	The data to act on: in RAG, the retrieved passages
Reasoning	Whether to think step by step before answering
Output format	The shape the answer must take
Examples	One or two input-output pairs showing style and depth
Delimiters	Keeps instructions, context and examples from bleeding together

Make it a template, not a blob

A reusable structure with placeholders filled at runtime. When you discover the summaries are pulling in quoted threads, you know exactly which section to change.

Version it like code

Store templates as files, commit them, and record the version in every trace alongside the model version.

Without it you cannot tell whether a behaviour change came from a prompt edit, a model update or a data shift.

Newer reasoning models generate their own chains of thought. Adding “think step by step” can interfere with that rather than help.

2.3 EVALUATION SETUPS

Every evaluation answers one of two questions

Absolute

Is this good enough to ship?

Reference-based: compare against a labelled answer. Clear right and wrong, but labels are expensive and many tasks have several valid outputs.

Reference-free: check against criteria you define. No gold answer needed, which is why most teams start here.

Comparative

Which of these is better?

Pairwise: two outputs for the same input, a judge picks one. Cheap, and good enough for early decisions.

Ranking and leaderboards: aggregate many comparisons into a system-level ordering.

Most teams' first bottleneck is getting **one** configuration to work reliably, so absolute evaluation comes first and this workshop spends most of its time there.

Even fifty to a hundred labelled examples pay for themselves: they validate your automated judges, catch regressions on known-good cases, and let you measure improvement over time. Error analysis produces them as a byproduct.

Start with the decision, not the metric

Goal	The question behind it	What it demands of the data	When
Set expectations	How good is this right now?	Small, broad, cheap: tens of examples	First prototype
Compare	Is B better than A?	Paired and identical across arms, enough to beat noise	Iteration
Decide	Ship, hold or roll back?	Representative of real traffic, thresholds agreed in advance	Pre-release
Assess risk	When does this go badly wrong?	Adversarial and edge cases, deliberately unrepresentative	Pre-release and ongoing
Diagnose	Why is it failing, and where?	Small, targeted, drawn from failures you have seen	After a bad result

Cost is almost always the cost of judgments. Metrics are free and compute is cheap. Somebody deciding whether an answer is right is the bottleneck, so price every goal in units of human attention.

Two evaluation objects, and the one nobody puts in scope

End-to-end

Question in, answer out. Reproduces the user's experience, so it is the only object that supports a deployment decision or a compliance claim.

Cannot tell you what to fix.

Components

Retriever, reranker, generator, and the corpus. Tells you where a failure originated.

Cannot support a claim about overall quality.

> *"Every component passed, therefore the system works"* does not follow. The interfaces between components are where meaning is lost.

Put the corpus in scope

Most teams exclude it because another function owns it, then find that a large share of their failures are corpus failures: stale, contradictory, missing, or never meant for that reader.

Excluding an object from evaluation does not exclude it from causing failures. It only removes your ability to name the cause.

"Thirty per cent of our failures trace to superseded documents" is a conversation the document owner can act on. "The AI is not working" is not.

WHENEVER SOMEONE SAYS “EVALUATE OUR RAG SYSTEM”

For exactly what task?

The input–output contract

One question or a conversation? Does it carry office, role, entitlement? Free text, text plus citations, or text plus a confidence signal?

Ground truth, two ways

Intensional: the rule for what counts as correct. **Extensional**: worked examples of right and wrong. You need both.

What the task is not

Helix does not give legal advice, interpret an individual contract, or discuss compensation. Stated explicitly, it bounds every later claim.

A refusal is an output, not the absence of one.

Leave it out of the contract and the evaluation scores every “I could not find that” as a failure, so the system gets tuned to guess. Correctly declining when the evidence does not support an answer is a capability worth measuring on its own.

Three questions the intensional rule has to settle: must **every** claim be supported, or only the material ones? If the answer exists in the world but not in the corpus, is answering it success or failure? When the corpus contradicts itself, what is correct behaviour?

There is no universally right answer. There is only a written one, and writing it is the work.

2.5 QUALITY DIMENSIONS FOR RAG

A dimension is what you assess. A metric is one way to measure it.

Layer	Dimensions	Observable from
Corpus	Coverage · recency · internal consistency · access correctness	The index and its metadata
Retrieval	Context relevance · context recall · ranking quality	Retrieved chunks against the question
Generation	Groundedness · answer correctness · answer relevance · completeness · citation correctness · appropriate refusal	Answer against context against reference
System	Latency · cost per query · robustness · fairness across offices and languages · injection resistance · entitlement leakage	Traces and production telemetry

Grouped by where they are observable, because that decides **who can act on them**. A corpus dimension is a conversation with a document owner, not a prompt change.

Cap your charter at five dimensions and rank them. A list of nine has prioritised nothing, and nine evaluators is a quarter of work nobody asked for.

THE DISTINCTION THE REST OF THE WEEK DEPENDS ON

Grounded and correct are independent

	CORRECT	NOT CORRECT
GROUNDED	**Working as intended** No intervention	**Faithful to a document that is wrong** The model did its job. The corpus did not. Fix ingestion and the corpus, not the prompt
NOT GROUNDED	**The dangerous one** Answered from pretraining, ignored the evidence. Wrong the moment your policy diverges from the open web, and end-to-end accuracy scores it as a win.	**Unsupported and wrong** Localise with the trace before guessing

Measured on the workshop corpus, three of six seeded questions land in **grounded and wrong**: faithful to their evidence, and wrong. A single “accuracy” number collapses the two and hides exactly this.

2.6 ACCEPTANCE CRITERIA

Write the thresholds before you measure

Not because the number will be right. It will not be. Because **a threshold written after seeing the result is a rationalisation.**

Target

Where you want this dimension to land.

Floor

Below which you would not ship, whatever the pressure that week.

Who agreed

A name. A threshold nobody agreed to is not a threshold.

Then a short **risk register**: for each plausible failure, its consequence, and whether anything currently detects it. Most rows will say *nothing* today. Filling that column is what the remaining three days are.

EXERCISE 2.A Alone, then compared in table groups

Your Evaluation Charter, v1

1 · Decision: what this evaluation supports, and who makes it

2 · Goal: which of the five archetypes, and why that one now

3 · Objects: what is in scope, including whether the corpus is

4 · Task: the input–output contract in one sentence, refusal included

5 · Ground truth: the rule, plus one correct and one incorrect example

6 · Not in scope: three things this system is not for

7 · Dimensions: at most five, ranked, one line of justification each

8 · Acceptance: target and floor for the top three, and who agreed

Field 5 takes the longest and causes the most disagreement inside a single team. **That disagreement is the deliverable.** Teams that never have it discover it tomorrow, inside their inter-annotator agreement scores.

Tomorrow opens by revising this. Module 05 turns fields 5 and 7 into rubrics; Module 06 turns field 8 into evaluator pass conditions.

03

RAG architecture, observability and traces

Ends at the keyboard

You cannot evaluate what you cannot see, and in a RAG system most of what decides answer quality happens in stages that produce no user-visible output at all.

3.1 RETRIEVAL FUNDAMENTALS

Start with keyword search

Keyword search: BM25

Ranks documents by the query terms they share. Fast, cheap, and you can always see *why* something was retrieved.

Surprisingly strong where vocabulary is stable: policies, tickets, product names

Misses anything phrased differently from the query

Embeddings

Turns text into vectors so that similar meanings sit close together. Matches “reset my login” to “can't sign in”.

Catches paraphrase that keywords cannot

Opaque: hard to say why a chunk ranked where it did

Hybrid

Runs both and fuses the rankings, commonly with reciprocal rank fusion.

Keywords catch exact terms embeddings miss; embeddings catch meaning keywords miss

Two systems to tune instead of one

Begin with the interpretable baseline. When you can see exactly which terms matched, debugging a bad answer takes minutes. Move to embeddings when you can show that keyword search is missing cases that matter, and not before.

3.2 THE FULL REQUEST PATH

Twelve stages. Only two of them involve the model.

BUILD TIME · RUNS ONCE, BEFORE ANY QUESTION IS ASKED

Ingestion	Parsing	Chunking	Enrichment	Embedding	Indexing
FP1 FP8 FP9	FP10	FP10	FP8	FP2	FP11

⬇ everything above reaches the user through one frozen artifact: the index

REQUEST TIME · RUNS FOR EVERY QUESTION

Query	Retrieval	Reranking	Assembly	Generation	Citation
filters	FP2 FP11	FP2	FP3	FP4 FP6 FP7	FP5

The answer

Every failure above is observed here, and only here, which is why an end-to-end score cannot tell you what to fix.

Same corpus, three ways to answer from it

VANILLA RAG · HELIX

A fixed path

```
query → retrieve → rerank
→ assemble → generate → cite
```

The pipeline decides. Every request takes the same six spans, whatever was asked.

Adds: FP1 to FP11, the catalog you build today

src/helix

AGENTIC RAG · HELIX AGENT

The model chooses the path

```
plan → call a tool → observe
loop until done or out of steps
tools: profile, office-filtered
search, document status
```

The model decides which tool, with which arguments, and when to stop.

Adds: tool choice, tool arguments, stopping, step budget: AG1 to AG5

src/agentic

GRAPHRAG · HELIX GRAPH

Evidence decided at build time

```
build: extract facts → graph
request: link → traverse
→ filter → rank → generate
```

An extractor decides once what a unit of evidence is: one row, with its office and role attached.

Adds: entity links, extraction, stale edges, lost labels, coverage: GR1 to GR5

src/graphrag

All three write the same trace schema. That is what lets one set of evaluators compare them on Day 3, and why today's trace fields matter beyond Helix.

3.3 THREE TERMS, USED PRECISELY FOR FOUR DAYS

Query, trace, span

Query

The raw input to the system. Often a person typing, but equally a scheduled job or another service calling your API.

Trace

The complete, ordered record of everything the system did in response: every message, every tool call, every value returned, and the final answer.

Span

One stage inside a trace, with its own timing, inputs and outputs. Critically, it also records what it discarded.

You evaluate traces, not individual steps.

An individual message can look perfectly reasonable in isolation and still lead to a wrong final answer. Reading the whole trace is what lets you find where the problem originated rather than mislabelling a later symptom as the cause.

Because outputs are not repeatable, one trace shows one possible execution. If a failure looks intermittent, run the same query several times and count.

3.4 WHERE FAILURES ARE BORN

A failure is observed at the answer and born somewhere upstream.

The distance between those two points is exactly why end-to-end metrics cannot tell you what to fix, and exactly what a trace closes.

Eleven failure points, and where each one is born

FP1 to FP7 catalogued from three production RAG deployments. FP8 to FP11 are what enterprise corpora add.

#	Failure point	Born at	What it looks like in the answer
FP1	Missing content	Corpus	A confident answer the documents cannot support
FP2	Missed top-ranked docs	Retrieval	It exists in the corpus but ranked below the cut-off
FP3	Not in context	Assembly	Retrieved, then dropped in consolidation or truncation
FP4	Not extracted	Generation	It was in the context and the model did not use it
FP5	Wrong format	Generation	Instructions about structure ignored
FP6	Incorrect specificity	Generation	Too general or too specific for what was asked
FP7	Incomplete	Generation	Part of the available answer omitted
FP8	Stale or superseded evidence	Ingestion	Faithful to a document that no longer reflects policy
FP9	Contradictory evidence	Corpus	Two valid-looking sources disagree; one is picked silently
FP10	Severed context	Chunking	The qualifier and the fact ended up in different chunks
FP11	Entitlement leakage	Retrieval	Content surfaced to a reader who should not see it

Seven of the eleven are born before the model is called. The shaded four are where enterprise systems spend most of their failure budget, and none of them is visible in the answer text alone. FP11 in particular will never show up in a faithfulness score: the answer is perfectly faithful to a document the reader was not entitled to see.

Agents and graphs add failure points of their own

FP1 to FP11 still apply: both architectures retrieve and generate. These are born in places vanilla RAG does not have.

AGENTIC RAG born in the step sequence

AG1 Missed tool call
the profile or status check never appears in the steps

AG2 Wrong tool argument
search ran with the asker's office, not the one asked about

AG3 Premature finish
fewer searches than the question has parts

AG4 Step budget exhausted
the loop hit max_steps before it could answer

AG5 Observation ignored
a page reported superseded is still cited

GRAPHRAG born at build time or at the link

GR1 Entity linking miss
“the Netherlands” never reaches the Amsterdam node

GR2 Extraction error
a row that lost its heading, written down as company-wide

GR3 Stale fact traversed
a fact from a superseded page reaches the answer

GR4 Access label lost
the fact says all; its source page says hr-only

GR5 Coverage gap
no fact on the topic, and nothing to fall back on

On Day 3 all three answer the same thirteen questions, scored by the same evaluators: vanilla 4, agent 7, graph 12 right. None wins every row. The agent runs out of steps comparing offices; the graph misses what no extraction rule captured.

3.5 THE MINIMUM VIABLE RAG TRACE

Six spans, and what each one has to record

```
rag.request   query_id · user hash · office · app version · config hash
 ├─ query.transform   original query · rewritten query · inferred filters
 ├─ retrieve   index · embedding model · top_k · filters
 |     → per candidate: chunk_id · doc_id · doc_version · last_modified · score
 ├─ rerank   model · input order · output order · cut-off
 |     → dropped: what did not survive
 ├─ context.assemble   final order · token budget · tokens used · truncation
 |     → dropped: what did not fit
 ├─ generate   model · prompt version · temperature · tokens · finish reason
 └─ postprocess   citations · guardrail verdicts · refusal flag
```

Span names follow the OpenTelemetry GenAI conventions, so the shape transfers to any observability platform. Teach the shape, not a vendor's SDK.

Three fields decide whether a trace is usable for evaluation

1 · Document version and last-modified date, on every retrieved chunk

Without them you cannot tell a grounded-but-wrong answer from an ungrounded one. Both look identical in the answer text.

2 · The dropped set, at every stage that discards something

Log what was retrieved *and* what did not survive reranking, deduplication and truncation. FP2 and FP3 live entirely in the difference between those two sets.

3 · The configuration hash

Chunk strategy, embedding model, top-k, prompt version, model version, hashed into one value. Without it, before and after are two piles of traces with no way to say what differed.

Source-to-answer lineage is what these three buy you: from any claim in the answer, reach the chunk, the document, and its version and date. Without it, every quality judgment is a judgment about *text*. With it, it becomes a judgment about *evidence*.

Your trace already holds what every toolkit reads

DeepEval and Ragas each read one test case per answer. Every field they need is already in the six traces you write today.

IN YOUR TRACE	DEEPEVAL	RAGAS
the question, and who asked	input	user_input
the answer	actual_output	response
chunks that reached the model	retrieval_context	retrieved_contexts
the reference answer	expected_output	reference
each agent step	tools_called	tool calls in messages

ONLY YOUR TRACE HOLDS

Document version and date
FP8: is the cited page current?

Access label
FP11: may this person read it?

What each stage dropped
FP2, FP3: where was the evidence lost?

No toolkit default reads these fields.

From tomorrow the toolkits join. They score these same traces beside the checks you build, and every score is compared with human labels before anyone trusts it. Hands-on H1 prints Sara's trace in this shape.

HANDS-ON H1

One script, run from VS Code

Instrument a pipeline, then read its traces

What you do

Run six seeded questions through Helix. Read one trace in full. Classify each failing trace against FP1–FP11, naming the span where it was born and the evidence in the trace that proves it. Then change one variable at a time and watch the failure move.

No evaluation library appears

You instrument before you measure, so that when a toolkit arrives tomorrow you already know what it is reading.

Four planted defects

FP8 / FP9: a superseded 2022 policy still in the index, disagreeing with the 2024 one
FP2: a notice-period table that ranks below the cut-off and never reaches the model
FP3: an answer paragraph that falls outside the context budget
FP11: a restricted document with no enforced access filter

Nobody is told which questions fail

Two of the six are clean.

All four failures look like ordinary answers. All four are unambiguous in the trace. That contrast is the whole argument for this module.

Runs live on your own model: Ollama on your laptop, or a cloud model with an API key. python run.py check tests the connection first. The figures on these slides are from the reference run (--offline); your live run will differ, and reading why is the exercise.

What you leave with, and what tomorrow does with it

Artifact	From	What Day 2 does with it
Evaluation Charter v1 eight fields	Exercise 2.A	Module 05 turns fields 5 and 7 into judgment rubrics; Module 06 turns field 8 into evaluator pass conditions
Six traces plus the triage worksheet	Hands-on H1	Module 04 uses them as the worked example for open coding, before you repeat it on your own
One-sentence gulf statement	Exercise 1.A	Opens Module 04: you check whether error analysis confirms it or overturns it

THE OVERNIGHT ASK

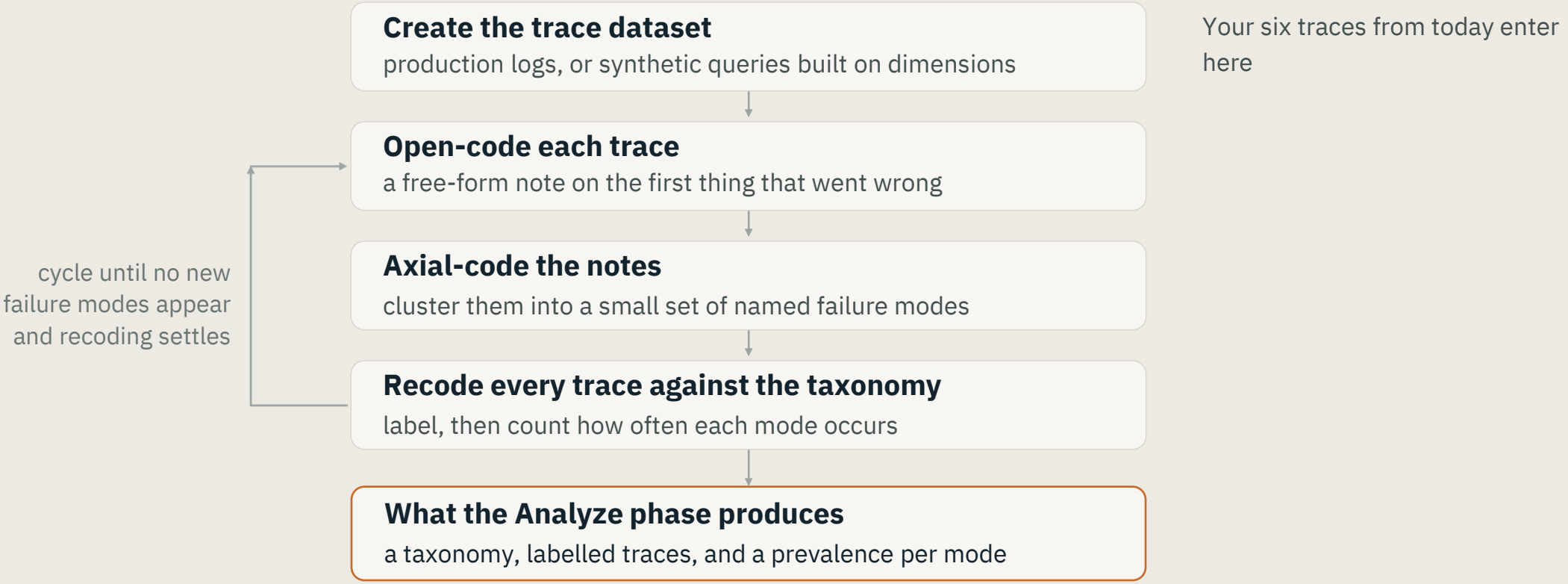
Bring twenty **real** queries from your own system. Production logs are ideal. No logs? Twenty questions you have actually heard users ask, from tickets, chat or a pilot.

Not twenty questions you think the system should handle well.

Self-authored evaluation questions cluster around what the author already knows the system does, which is the generalization gulf reproducing itself inside the evaluation set.

WHAT DAY 2 MORNING ACTUALLY IS

Error analysis is a loop, not a pass



A model may help cluster notes you have already written. It must not propose the taxonomy before anyone has read a trace.

Close · Day 2 preview · Sources: Husain, A field guide to rapidly improving AI products · Orosz and Husain, The Pragmatic Engineer

Before tomorrow

These four are the ones worth reading tonight. Every slide's footer links to its sources.

Seven Failure Points When Engineering a Retrieval Augmented Generation System
Barnett, Kurniawan, Thudumu, Brannelly and Abdelrazek · CAIN 2024 · FP1 to FP7, from three production deployments

Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools
Magesh, Surani, Dahl, Suzgun, Manning and Ho · Journal of Empirical Legal Studies, 2025 · the 17–33% study and its failure typology

Lost in the Middle: How Language Models Use Long Contexts
Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni and Liang · TACL 2024 · why context assembly order is a design variable

Who Validates the Validators?
Shankar, Zamfirescu-Pereira, Hartmann, Parameswaran and Arawjo · UIST 2024 · criteria drift, and why your charter is a v1

Exit check. What decision is your evaluation supposed to support, and who makes it? Name two dimensions that matter for your system and one that does not. Given a wrong answer, name two places the failure could have been born and the trace field that would tell them apart.